



**Escuela Politécnica Nacional
Facultad de Ingeniería de Sistemas
Construcción y Evolución de Software**



Integrantes: Nahomy Llumiquinga
Brandon Pallo
Luis Villa

Fecha: 13/04/2026

Taller práctico: Desarmando Gigantes - Ingeniería Inversa en la Práctica

Fase 1:

Ingeniería Inversa

La ingeniería inversa es básicamente “ir hacia atrás” en un software. Ver cómo está hecho por dentro cuando no se dispone de toda la información. Es desarmarlo para entender su lógica, su estructura y por qué funciona así.

Mantenimiento de Software

Es todo lo que se hace después de que el software ya está funcionando. El mantenimiento puede ser: correctivo (arreglar errores), adaptativo (ajustarlo a cambios, como nuevos sistemas o reglas), perfectivo (mejorarlo o agregar cosas nuevas) o preventivo (evitar problemas futuros)

La ingeniería inversa es importante aquí porque muchas veces no hay documentación, entonces se debe entender el sistema desde el código para poder modificarlo sin dañarlo.

Evolución del Software

Es cómo un sistema va cambiando con el tiempo para no quedarse obsoleto. La ingeniería inversa ayuda porque permite entender sistemas antiguos y actualizarlos, mejorarlos o adaptarlos sin tener que empezar desde cero.

Fase 2:

1. El Pasado: La Era del Binario y el "Low-Level" (Años 90)

En esta época, el software era mayoritariamente monolítico y local (se instalaba desde un CD o disquete). El reto era entender archivos .exe o .bin cerrados.

Técnicas:

- **Desensamblado:** Convertir el código máquina (0101) en lenguaje Assembly. Era como leer planos de una casa ladrillo por ladrillo.
- **Depuración (Debugging) en tiempo real:** Ejecutar el programa paso a paso para ver qué pasaba en los registros de la CPU y la memoria RAM.

Herramientas Icónicas:

- **IDA Pro:** El estándar de oro (aún vigente) para visualización de grafos de control.
- **SoftICE / OllyDbg:** Permitían "congelar" el sistema operativo para analizar qué hacía un programa justo antes de pedir una contraseña o licencia.

2. El Presente: Cloud, Microservicios y Ofuscación

Hoy, el software no está en un solo archivo; está distribuido en la nube. La ingeniería inversa ya no se trata solo de leer código, sino de entender flujos de datos.

El Reto de la Ofuscación: Los desarrolladores usan herramientas para que el código sea ilegible (cambian nombres de variables login a l1). La ingeniería inversa moderna debe "limpiar" este ruido.

Técnicas en Microservicios:

- **API Sniffing:** Como el código corre en servidores ajenos, analizamos las peticiones JSON que viajan por la red para deducir la lógica del servidor.
- **Contenedores:** Realizar ingeniería inversa a imágenes de Docker para entender cómo están configuradas las dependencias y la infraestructura.

Herramientas Modernas:

- **Ghidra:** Desarrollada por la NSA, permite descompilar (pasar de binario a C casi legible) de forma gratuita.
- **Jadx / Frida:** Específicas para Apps móviles (Android), permitiendo modificar el comportamiento de una App mientras se ejecuta.

3. El Futuro: Inteligencia Artificial y Automatización

La IA está cambiando las reglas del juego, convirtiendo procesos que antes tomaban semanas en tareas de segundos.

- **Decompilación Asistida por LLMs:** Herramientas que toman un bloque de código en Assembly (ilegible para humanos) y le piden a un modelo como Gemini: "Explicame qué hace esta función en lenguaje natural". La IA es excelente identificando patrones de algoritmos conocidos.
- **Análisis de Vulnerabilidades Automático:** IAs que escanean binarios en busca de fallos de seguridad (Buffer Overflows) sin necesidad de que un humano lea el código.
- **De-ofuscación Inteligente:** Algoritmos que "adivinan" los nombres originales de las variables basándose en el contexto de lo que hace la función.

Fase 3:

Objetivo: Entender el funcionamiento del algoritmo de recomendaciones de Netflix

Utilizaríamos la técnica API sniffing para poder interceptar las peticiones para interceptar y capturar el tráfico de datos. La conversación entre los dispositivos y los servidores de Netflix usando una PROXY. Interceptaríamos peticiones HTTP y HTTPS para ver los datos crudos que se envía desde el celular para poder entender que tipo de información necesita el algoritmo para la recomendación.

También se podría descargar el .apk y usar herramientas como Jadx para descompilarlo y poder leer el código fuente para así poder leer su funcionamiento.

Aplicaríamos prueba de casa, lo haríamos mediante prueba en tiempo real viendo un tipo de contenido y revisando que tipo de datos se envía y cuál es la recompensa por cada tipo de contenido y acción que se toma. De esta manera podemos ver como cambian las recomendaciones según el tiempo que vemos y el tipo de contenido que vemos.



Prompt: “Dada esta información, genera una imagen, donde se muestre la "ingeniería inversa" en Netflix.”